Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине «Конструирование программного обеспечения»
Тема: «Создание основы проекта»

Выполнил:
Студент 2 курса
250966
Группы ПО-17
Фещук А.А.
Проверила:
Кулик А.Д.

Брест 2026

**Цель работы:** создать проект, который заложит основу для последующих лабораторных работ, а также позволит отобразить главное меню приложения.

4. Создание игры «*Город мечты*». В настройках должны быть *параметры размера города*, *начального бюджета* и *количества жителей*. Каждое значение в настройке должно переключаться на следующее значение при выборе его в подменю (например, бюджет может принимать значения 1000, 5000 и 10000 и при выборе этого значения настройки оно будет меняться по правилу 1000->5000->10000->1000…)

## Ход работы

```
#include <iostream>
#include <string>
using namespace std;
const int MENU_START_GAME = 1;
const int MENU_LOAD_GAME  = 2;
const int MENU_SETTINGS   = 3;
const int MENU_ABOUT      = 4;
const int MENU_EXIT       = 5;
const int SET_CITY_SIZE   = 1;
const int SET_BUDGET      = 2;
const int SET_POPULATION  = 3;
const int SET_BACK        = 4;

struct GameSettings {
    int citySize;
    int budget;
    int population;

    GameSettings() : citySize(10), budget(1000), population(100) {}
};

void clearScreen() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
```

```
}

void waitForEnter() {
    cout << "\nНажмите Enter, чтобы вернуться в меню...";
    cin.ignore(10000, '\n');
    cin.get();
}

void showMainMenu() {
    cout << "====================================\n";
    cout << "          ГОРОД МЕЧТЫ\n";
    cout << "====================================\n";
    cout << MENU_START_GAME << ". Начать игру\n";
    cout << MENU_LOAD_GAME  << ". Загрузить игру\n";
    cout << MENU_SETTINGS   << ". Настройки\n";
    cout << MENU_ABOUT      << ". О программе\n";
    cout << MENU_EXIT       << ". Выход\n";
    cout << "====================================\n";
    cout << "Выберите пункт меню: ";
}

void startGame() {
    clearScreen();
    cout << "=== НАЧАЛО ИГРЫ ===\n";
    cout << "Вы начали новую игру в «Город мечты»!\n";
    waitForEnter();
}

void loadGame() {
    clearScreen();
    cout << "=== ЗАГРУЗКА ИГРЫ ===\n";
    cout << "Данная функция на данный момент недоступна\n";
    cout << "и появится в ближайших обновлениях.\n";
    waitForEnter();
}

void switchCitySize(int &size) {
    if (size == 10)      size = 20;
    else if (size == 20) size = 30;
    else                 size = 10;
```

```
}

void switchBudget(int &budget) {
    if (budget == 1000)      budget = 5000;
    else if (budget == 5000) budget = 10000;
    else                     budget = 1000;
}

void switchPopulation(int &population) {
    if (population == 100)      population = 500;
    else if (population == 500) population = 1000;
    else                        population = 100;
}

void showSettingsMenu(const GameSettings &settings) {
    cout << "====================================\n";
    cout << "             НАСТРОЙКИ\n";
    cout << "====================================\n";
    cout << SET_CITY_SIZE  << ". Размер города:      " <<
settings.citySize   << "\n";
    cout << SET_BUDGET     << ". Начальный бюджет:   " <<
settings.budget     << "\n";
    cout << SET_POPULATION << ". Количество жителей: " <<
settings.population << "\n";
    cout << SET_BACK       << ". Назад\n";
    cout << "====================================\n";
    cout << "Выберите пункт: ";
}

void settingsMenu(GameSettings &settings) {
    int choice = 0;
    while (choice != SET_BACK) {
        clearScreen();
        showSettingsMenu(settings);
        cin >> choice;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(10000, '\n');
            continue;
        }
```

```
        switch (choice) {
            case SET_CITY_SIZE:
                switchCitySize(settings.citySize);
                break;
            case SET_BUDGET:
                switchBudget(settings.budget);
                break;
            case SET_POPULATION:
                switchPopulation(settings.population);
                break;
            case SET_BACK:
                break;
            default:
                cout << "Неверный пункт меню!\n";
                waitForEnter();
                break;
        }
    }
}

void showAbout() {
    clearScreen();
    cout << "=== О ПРОГРАММЕ ===\n";
    cout << "Игра: «Город мечты»\n";
    cout << "Версия программы: 1.0\n";
    cout << "Студент: Фещук Артём\n";
    waitForEnter();
}

int main() {
    GameSettings settings;
    int choice = 0;

    while (choice != MENU_EXIT) {
        clearScreen();
        showMainMenu();
        cin >> choice;

        if (cin.fail()) {
```

```
            cin.clear();
            cin.ignore(10000, '\n');
            continue;
        }

        switch (choice) {
            case MENU_START_GAME:
                startGame();
                break;
            case MENU_LOAD_GAME:
                loadGame();
                break;
            case MENU_SETTINGS:
                settingsMenu(settings);
                break;
            case MENU_ABOUT:
                showAbout();
                break;
            case MENU_EXIT:
                cout << "Выход из программы...\n";
                break;
            default:
                cout << "Неверный пункт меню!\n";
                waitForEnter();
                break;
        }
    }

    return 0;
}
```

## Контрольные вопросы

**0. Алгоритм работы написанной программы**

Словесное описание:

Начало. Программа запускается, создаётся объект GameSettings со значениями по умолчанию (размер города = 10, бюджет = 1000, жителей = 100).

Цикл главного меню. Пока пользователь не выбрал «Выход»:

Очищается экран.

Выводится главное меню с 5 пунктами.

Считывается номер пункта.

Если ввод некорректен (не число) - сбрасывается состояние cin и цикл повторяется.

Иначе выполняется switch по выбранному пункту:

1 → startGame(): сообщение о начале игры → ожидание Enter → возврат в меню.

2 → loadGame(): сообщение о недоступности → ожидание Enter → возврат в меню.

3 → settingsMenu(settings): вход в подменю настроек.

4 → showAbout(): вывод версии и ФИО студента → ожидание Enter → возврат в меню.

5 → выход из цикла, завершение программы.

другое → сообщение об ошибке → возврат в меню.

Подменю настроек. Пока пользователь не выбрал «Назад»:

Выводятся текущие значения параметров.

Считывается номер пункта.

1 - размер города переключается по кругу 10 → 20 → 30 → 10.

2 - бюджет переключается 1000 → 5000 → 10000 → 1000.

3 - жители переключаются 100 → 500 → 1000 → 100.

4 - возврат в главное меню.

Изменения сохраняются в структуре, поэтому при повторном входе значения не сбрасываются.

Конец. При выборе «Выход» программа завершает работу с кодом 0.

**1. Основные подходы к проектированию программ**

**Структурное (процедурное) программирование.** Программа разбивается на отдельные функции, каждая из которых решает свою подзадачу. Логика строится по принципу «сверху вниз»: сначала определяется общая структура, затем каждая часть детализируется. Применяется в небольших проектах, учебных задачах, консольных утилитах.

**Модульное программирование.** Проект делится на независимые модули, каждый из которых имеет чёткий интерфейс (что подаётся на вход, что возвращается). Модули можно разрабатывать и тестировать по отдельности. Используется в средних и крупных проектах, где важно разделение труда и повторное использование кода.

**Объектно-ориентированное проектирование (ООП).** Мир моделируется классами и объектами. Основные принципы - инкапсуляция, наследование, полиморфизм. Позволяет описывать сущности предметной области (например, «Игрок», «Город», «Настройки») и их взаимодействие.

**Функциональное программирование.** Вычисления представляются как композиция чистых функций без побочных эффектов. Данные не изменяются, а преобразуются. Применяется в задачах обработки данных, параллельных вычислениях, там, где важна предсказуемость и отсутствие ошибок из-за общего состояния.

## 2. Основные этапы жизненного цикла проекта

Классическая модель (каскадная, Waterfall) включает:

1. Анализ требований - выяснение, что должна делать программа, сбор пожеланий заказчика, формирование ТЗ.

2. Проектирование - выбор архитектуры, разбиение на модули, проектирование интерфейсов и структур данных.

3. Реализация (кодирование) — написание исходного кода, компиляция.

4. Тестирование - проверка корректности: модульное, интеграционное, системное, приёмочное.

5. Внедрение (развёртывание) - установка и передача заказчику.

6. Сопровождение - исправление ошибок, доработка, обновление версий.

## 3. Оценка стоимости ошибок

Стоимость ошибки - это суммарные затраты на её обнаружение и исправление. Чем позже обнаружена ошибка, тем дороже она стоит.

| Этап обнаружения | Относительная стоимость исправления |
|---|---|
| Анализ требований | 1× |
| Проектирование | 3–5× |
| Кодирование | 10× |
| Тестирование | 20–50× |
| Внедрение | 50–100× |
| Сопровождение (у заказчика) | 100–1000× |

**Причины роста стоимости:**

- Ошибку нужно сначала найти (отладка, логи, репро).
- Приходится менять уже написанный и протестированный код.
- Могут быть затронуты другие модули (эффект домино).
- Если продукт уже у заказчика - нужны патчи, повторное тестирование, возможны репутационные потери.

**4. Требования к программному продукту**
Требования делятся на функциональные и нефункциональные.
Функциональные требования (что делает программа)
**Для нашей лабораторной:**

- Программа отображает главное меню с 5 пунктами.
- Ввод номера пункта осуществляется с клавиатуры.
- Номера пунктов вынесены в константы.
- Пункт «Начать игру» выводит сообщение о начале игры.
- Пункт «Загрузить игру» выводит сообщение о недоступности.
- Пункт «Настройки» открывает подменю с параметрами варианта:
- размер города,
- начальный бюджет,
- количество жителей.
- Значения в настройках переключаются по кругу и сохраняются между заходами.
- Пункт «О программе» выводит версию и ФИО студента.
- Пункт «Выход» завершает работу программы.

**5. Проектирование пользовательского интерфейса**
Основные принципы (для консольного приложения):

1. Простота и понятность. Пользователь сразу видит, что можно сделать. В нашей программе - нумерованное меню, где номер соответствует действию.

2. Единообразие. Все меню оформлены одинаково: заголовок, список пунктов, строка приглашения. Один и тот же стиль рамок и разделителей.

3. Обратная связь. После каждого действия пользователь видит результат (сообщение) и понимает, как вернуться назад (подсказка «Нажмите Enter...»).

4. Защита от ошибок. Неверный ввод не ломает программу: выводится сообщение, и пользователь возвращается в меню. Используется сброс cin при ошибке ввода.

5. Минимум усилий. Не нужно вводить длинные команды - достаточно одной цифры.

6. Логичная структура. Главное меню → подменю настроек → возврат. Пользователь всегда знает, где он находится.

7. Константы вместо «магических чисел». Пункты меню заданы константами (MENU_START_GAME и т.д.), что упрощает чтение и изменение кода.

8. Визуальное разделение. Рамки из = отделяют меню от остального вывода, заголовки выделены.